

Introduction to Computing

Lecture 1

CS 1001

Kirk Duran

Welcome

Key idea

This course is about learning how computation works, and how to use it to solve problems.

- ▶ Instructor: Kirk Duran
- ▶ Course: CS 1001, Introduction to Computing
- ▶ Unit: 1. Computing and Program Execution
- ▶ Slides, examples, and assignments will live on Canvas.



About Me

- ▶ Full-Time Lecturer in Computer Science and Software Engineering.
- ▶ Teaching interests: introductory computing, data structures, object-oriented design, algorithms, and artificial intelligence.
- ▶ Research interests: deep reinforcement learning, heuristic search, robotics, and computer science education.



About Me: Path

- ▶ Grew up in Boston.
- ▶ Went to high school in North Carolina.
- ▶ Started at community college in Los Angeles.
- ▶ Finished at UC Berkeley in applied mathematics and computer science.
- ▶ Earned an M.S. in computer engineering from Sonoma State.
- ▶ Spent time in doctoral study in computer science at UNH.
- ▶ Worked in Silicon Valley before coming to Cal Poly.

About Me: Work And Life

Professional

Robotics engineering at Physik Instrumente, machine learning engineering at Remi AI, and teaching/mentoring at Cal Poly.



Mentoring

Beacons, OUDI, CENG SURP, projects, resumes, interview prep, and letters.

Personal

Basketball, weightlifting, video games, anime, and trying to make technical ideas feel less mysterious. I am here to help you learn the material and navigate the field.

Today

- ▶ What computer science studies.
- ▶ Why computing exists as a discipline.
- ▶ Why computing matters even if you do not become a software engineer.
- ▶ How this course will work.
- ▶ What to set up before next time.

Part 1: What Are We Studying?

Two horizontal lines are positioned below the title. The top line is orange and shorter, while the bottom line is white and longer, extending across most of the slide width.

What does this even mean?

A Misleading Name

Key idea

“Computer science” can sound like the subject is mostly about computers.

Example

That is a little like calling mathematics “calculator studies.”

- ▶ Computers matter because they carry out computation.
- ▶ The deeper subject is how we represent problems and design precise processes.

Start With Computation

Definition

Computation is the process of transforming information according to precise rules.

Example

A recipe, a payroll calculation, a route planner, a search engine, and a video game all describe processes over information.

Computer Science

Definition

Computer science is the study of computation: how information can be represented, transformed, stored, communicated, and used to solve problems.

- ▶ A computer is one tool for carrying out computation.
- ▶ A program is a precise description of steps.
- ▶ A programming language gives us a way to write those steps for a machine and a human reader.

A Useful Mental Model

Input

Data we start with.

Process

Rules we follow.

Output

Result we produce.

In class

Pick one everyday task and identify its inputs, process, and outputs.

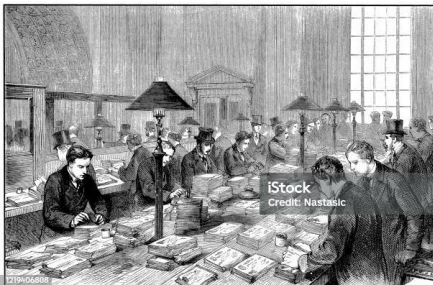
Part 2: Why Does Computer Science Exist?

Two horizontal lines are positioned below the title. The top line is orange and shorter, while the bottom line is white and longer, extending further across the slide.

Why Does Computer Science Exist?

Let's hear from you.

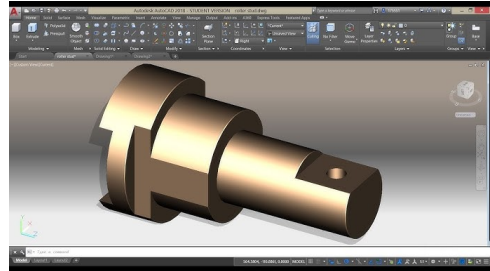
Paper Work At Scale



Key idea

A lot of early computing was about replacing or amplifying clerical labor: sorting, counting, tabulating, payroll, inventory, census work, and records.

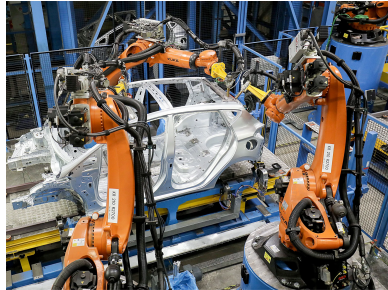
Drawing Work At Scale



Key idea

Computation turned drawings into editable, shareable models: design became something we could copy, simulate, revise, and manufacture from.

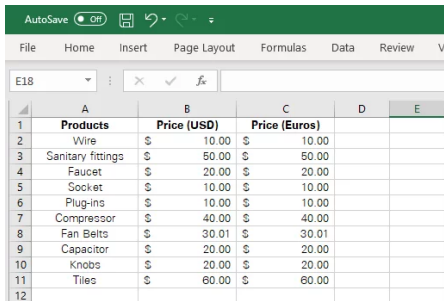
Production Work At Scale



Key idea

Software does not only live on laptops. It coordinates machines, sensors, logistics, schedules, safety checks, and feedback loops.

Everyday Models At Scale



The screenshot shows a Microsoft Excel interface. The top ribbon includes 'AutoSave' (Off), 'File', 'Home', 'Insert', 'Page Layout', 'Formulas', 'Data', and 'Review'. The active cell is E18. The spreadsheet contains a table with the following data:

	A	B	C	D	E
1	Products	Price (USD)	Price (Euros)		
2	Wire	\$ 10.00	\$ 10.00		
3	Sanitary fittings	\$ 50.00	\$ 50.00		
4	Faucet	\$ 20.00	\$ 20.00		
5	Socket	\$ 10.00	\$ 10.00		
6	Plug-ins	\$ 10.00	\$ 10.00		
7	Compressor	\$ 40.00	\$ 40.00		
8	Fan Belts	\$ 30.01	\$ 30.01		
9	Capacitor	\$ 20.00	\$ 20.00		
10	Knobs	\$ 20.00	\$ 20.00		
11	Tiles	\$ 60.00	\$ 60.00		
12					

Key idea

Spreadsheets made computation feel ordinary: not just for scientists, but for budgets, schedules, grades, inventory, experiments, and decisions.

- ▶ A table can become a model.
- ▶ A formula can become a rule.
- ▶ A change can ripple through a whole system.

Reason 1: Scale

Key idea

Computers let us perform simple rules many times, quickly and consistently.

- ▶ Search millions of records.
- ▶ Simulate weather, traffic, or molecules.
- ▶ Coordinate communication across the world.

Reason 2: Precision

Key idea

Programming forces vague ideas to become precise procedures.

Example

“Sort these names” sounds simple until we ask: alphabetical by first name or last name? What about accents? What about ties? What about missing data?

Reason 3: Leverage

Key idea

A small program can affect a large number of people.

- ▶ That can be wonderful: accessibility tools, medical analysis, education, art.
- ▶ That can be dangerous: bias, privacy failures, brittle automation.
- ▶ This is why we practice both technical care and social responsibility.

Economic Impact

Key idea

Each wave of computing turned a technical idea into a new economic platform.

Long arc

Mainframes, personal computers, the web, mobile/cloud, and now AI all changed what firms can measure, automate, sell, and scale.

Current scale

BEA estimated that digital economy real value added grew 6.3% in 2022, compared with 1.9% for total U.S. real GDP.

Public markets

S&P lists Information Technology at 36.8% of the S&P 500 by index weight as of July 31, 2026.

The Impact Of Computer Science

In class

Think of one system you used this week that made a decision about you, for you, or near you.

- ▶ What information did it use?
- ▶ What did it output?
- ▶ Who benefited?
- ▶ Who could be harmed if it failed?

The Point

Key idea

Computer science is not only about computers. It is about building precise tools for messy human problems.

- ▶ We need logic.
- ▶ We need creativity.
- ▶ We need communication.
- ▶ We need judgment.

Part 3: Why Study Computer Science?

Two horizontal lines are positioned below the title. The top line is orange and shorter, while the bottom line is white and longer, extending across most of the slide width.

Why Study Computer Science?

Let's hear from around the room.

A Practical Answer

- ▶ Computing is part of nearly every discipline.
- ▶ Programming helps you automate work and analyze information.
- ▶ Reading code helps you understand the systems around you.
- ▶ Debugging builds a general problem-solving muscle.

A Deeper Answer

Key idea

You are learning a new way to think: break a problem into pieces, make assumptions explicit, test your reasoning, and revise.

Example

This way of thinking transfers beyond Python: spreadsheets, data analysis, research, design, engineering, operations, and everyday troubleshooting.

Who Counts As A Computer Scientist?

Key idea

The people who shaped computing do not all fit one narrow stereotype.



Grace Hopper
Navy officer, compiler pioneer; made programming more usable.



Katherine Johnson
NASA mathematician and “human computer” for spaceflight.



Richard Tapia
First-generation Mexican American mathematician; optimization and mentoring.



Alan Turing
Mathematician/logician; theory, cryptanalysis, and machines.

Why These Examples Matter

Example

They seem “nontraditional” only if the stereotype is too narrow.

- ▶ Hopper: Navy officer who argued programming should be closer to human language.
- ▶ Johnson: mathematical labor before electronic machines took over much of that work.
- ▶ Tapia: first-generation path and mentoring that widened who belongs.
- ▶ Turing: logic and mathematics before “software engineer” was a job title.

Belonging In Computer Science

Key idea

Maybe the problem is not that students fail to match the “traditional” CS type. Maybe the stereotype is wrong.

- ▶ Computing has always been built by people with different paths, skills, identities, and motivations.
- ▶ The useful question is not “Do I look like a computer scientist?”
- ▶ The useful question is “What problems can I learn to reason about?”

Community



Color Coded by ColorStack

Cal Poly has a supportive community for students interested in computing, belonging, professional development, and peer support.

- ▶ Join the Discord.
- ▶ Watch for events and resources.
- ▶ Bring a friend.

Part 4: How This Course Works



What We Practice In CS 1001

- ▶ Reading code carefully.
- ▶ Predicting what a program will do before running it.
- ▶ Writing small programs that solve clear problems.
- ▶ Testing, debugging, and explaining your reasoning.

Key idea

The goal is not memorizing Python syntax. The goal is building a durable way to think.

How The Course Will Feel

Early weeks

Expressions, variables, functions, conditionals, and tracing.

Later weeks

Objects, algorithms, files, larger programs, and applications.

Middle weeks

Lists, strings, mutation, loops, dictionaries, and structured data.

All quarter

Practice, feedback, persistence, and small wins that compound.

The Reality Check

- ▶ We start slowly on purpose.
- ▶ The pace will increase as ideas combine.
- ▶ The hard part is usually not typing code; it is reasoning about state, order, and edge cases.
- ▶ Waiting until the night before makes programming feel much harder than it needs to.

In class

The move: start early, get stuck early, ask early.

Student Expectations

Math

Comfort with arithmetic and basic algebra helps with calculations, variables, and logical structure.

Attention to detail

Small differences matter: names, parentheses, indentation, punctuation, and order.

Communication

You will explain what your code does and why it works.

More Expectations

Problem solving

Break tasks into steps, test assumptions, and revise your plan when reality disagrees.

Time management

Programming rewards steady practice. Short repeated sessions beat heroic last-minute sessions.

Persistence

Confusion is not failure. It is the normal starting point for learning a technical skill.

Canvas And Course Materials

- ▶ Canvas will host announcements, assignments, grades, and due dates.
- ▶ Public slides will also be available from the course website.
- ▶ Example code will be posted when it helps you practice.

In class

Open Canvas, find this course, find today's lecture slides, and open the syllabus.

Part 5: First Technical Setup



Technical Skill: Search

Key idea

Being able to search well is part of being able to program well.

1. Search for Google Colab.
2. Open a new notebook.
3. Search for how to print `Hello, world!` in Python.

Technical Skill: Typing

- ▶ Comfortable typing makes practice less frustrating.
- ▶ You do not need to be fast today.
- ▶ Accuracy matters more than speed.

Example

Optional practice site: <https://monkeytype.com/>

Technical Skill: Copy, Paste, Run

1. Copy a small Python `Hello, world!` example.
2. Paste it into your Colab notebook.
3. Click the run button.
4. Change the message and run it again.

Key idea

Running code is a conversation with the machine. Change one thing, observe what changes.

Technical Skill: File Management

1. Create a folder somewhere on your computer.
2. Give it a boring, obvious name: CS1001, Fall_CS1001, or IntroComputing.
3. Save your first notebook or file there.
4. Make sure you can find it again.

Part 6: The Sales Pitch



My Commitment: Beyond Syntax

- ▶ We are not just memorizing Python.
- ▶ We are learning the underlying mechanisms of computing.
- ▶ By the end, you should have logic that transfers to other languages and tools.
- ▶ This course is foundation for future computing work.

The Scientist's Mindset

- ▶ I will ask you questions.
- ▶ It is okay to say “I do not know.”
- ▶ Our job is not to know instantly.
- ▶ Our job is to figure things out carefully.

In class

When you get stuck, say what you expected, what happened, and what you tried.

Before Next Time

- ▶ Read the syllabus.
- ▶ Make sure you can access Canvas.
- ▶ Create your course folder.
- ▶ Run `print("Hello, world!")` in Python.
- ▶ Bring one question about computers, programs, or the course.

Key idea

Next time: how computers execute programs.

Next Time

How Computers Execute Programs